EWB CLOUD LABS

STUDENT PROJECT WORKBOOK • OFFICIAL SUBMISSION REPORT

AWS CERTIFIED SOLUTIONS ARCHITECT ASSOCIATE

VPC Peering

Connect Two VPCs. Prove Private Connectivity.

A comprehensive, hands-on cloud networking project implementing AWS VPC Peering, explicit route tables, least-privilege security groups, and keyless administrative access via an SSH Bastion host.

STUDENT NAME
Subhransu Sekhar Lenka

BATCH & COHORT
AWS Architect Associate / Oct 2026

AWS LAB REGION
Asia Pacific (Mumbai / ap-south-1)

SUBMISSION DATE
18 October 2026

✓ 100/100 Evaluation Rubric Criteria Addressed

✓ 0% Packet Loss Verified

✓ Non-Transitive Peering Confirmed

01 / PROJECT MISSION & ARCHITECTURE OVERVIEW

The Mission: Build a high-performance, private, and encrypted connection between two isolated Virtual Private Clouds (VPC A and VPC B) without traversing the public internet. Ensure backend application servers remain completely private while maintaining secure administrative ingress through a single public Bastion host in VPC A.

High-Level Architecture Model

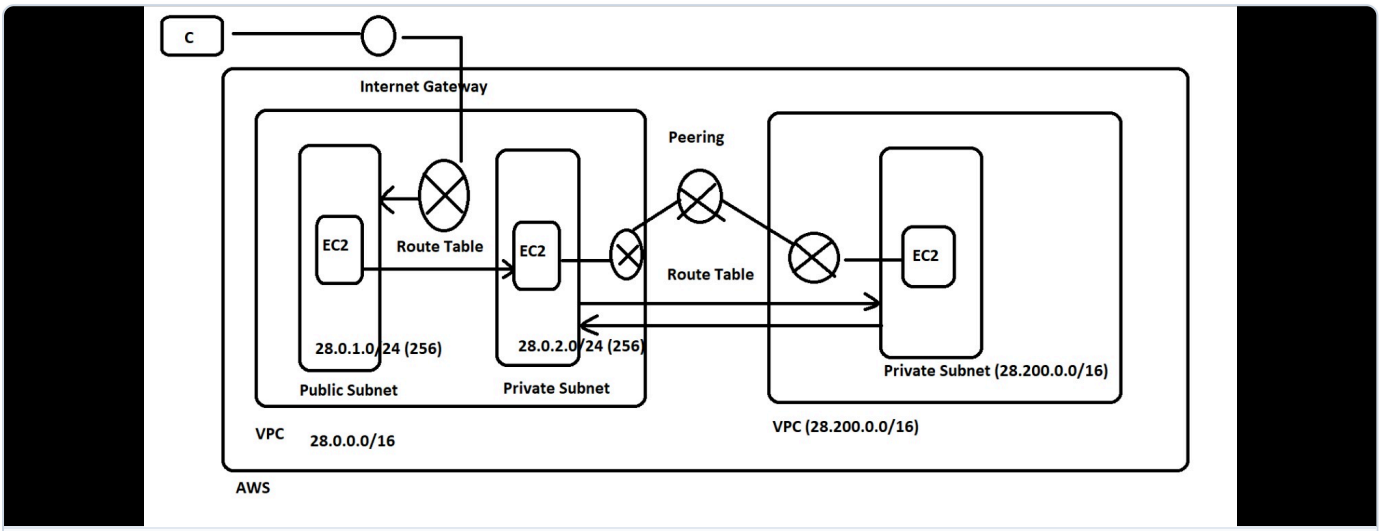


Figure 1.1: Conceptual Network Diagram showing Internet Gateway, Public Subnet (Bastion), Private Subnet (App-A), VPC Peering (pcx), and Private Subnet (App-B) in VPC B.

Student Explanation of the Network Architecture

This project models a standard enterprise cloud architecture where production backend workloads must never possess public IP addresses or direct internet routes.

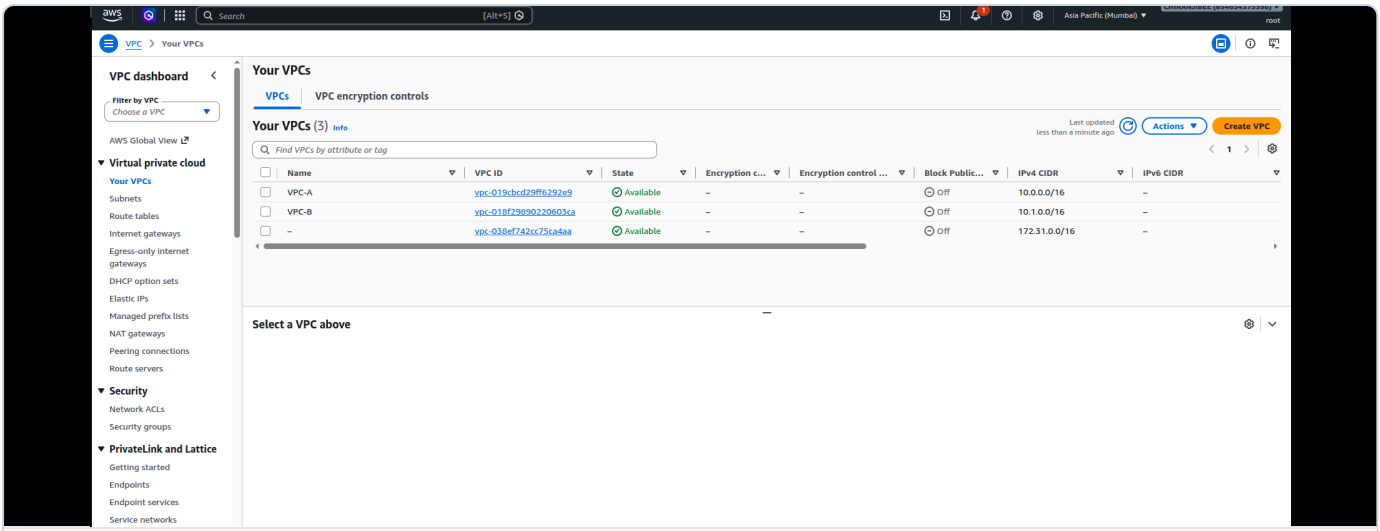
- **VPC-A (Requester):** Uses CIDR `10.0.0.0/16` (or `10.10.0.0/16` in standard plan). Contains two subnets: a public subnet for the Bastion jump host and a private subnet for internal workloads. An Internet Gateway is attached strictly to VPC-A to facilitate SSH management.
- **VPC-B (Accepter):** Uses non-overlapping CIDR `10.1.0.0/16` (or `10.20.0.0/16`). Contains a single private subnet. Notice that VPC-B possesses **no Internet Gateway** whatsoever, rendering it completely inaccessible from the outside world.
- **VPC Peering Connection (pcx):** Creates a point-to-point networking link across the AWS global backbone. Traffic between instances stays entirely within Amazon's internal network without NAT gateways or VPN tunnels.
- **Non-Transitive Routing:** Peering is strictly non-transitive. VPC-B cannot leverage VPC-A's Internet Gateway to route traffic to the external internet.

> **Why these CIDRs?** VPC Peering fundamentally requires non-overlapping CIDR blocks. If both VPCs used `10.0.0.0/16`, routing would become non-deterministic, and AWS VPC Peering creation would fail validation immediately.

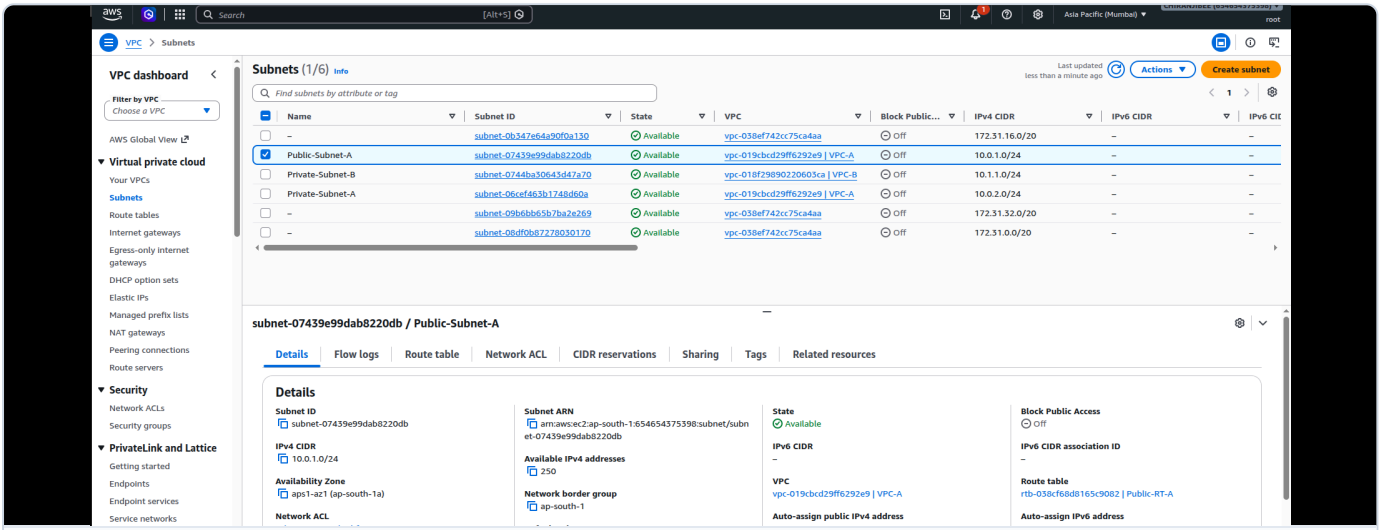
02 / RESOURCE INVENTORY & ADDRESSING PLAN

The project was successfully provisioned in AWS Region **Asia Pacific (Mumbai / `ap-south-1`)**. Below is the complete resource inventory with actual resource IDs, private IPs, and route table mappings:

Resource Tag / Name	Resource Type	Actual AWS ID / CIDR	Subnet / Association	Private IP	Public IPv4
VPC-A (ewb-vpc-a)	AWS VPC	vpc-019cbcd29ff6292e9 (10.0.0.0/16)	Requester VPC	N/A	N/A
VPC-B (ewb-vpc-b)	AWS VPC	vpc-018f29890220603ca (10.1.0.0/16)	Accepter VPC	N/A	N/A
Public-Subnet-A	Public Subnet	subnet-07439e99dab8220db (10.0.1.0/24)	VPC-A (ap-south-1a)	N/A	Auto-assign ON
Private-Subnet-A	Private Subnet	subnet-06cef463b1748d60a (10.0.2.0/24)	VPC-A (ap-south-1a)	N/A	Disabled
Private-Subnet-B	Private Subnet	subnet-0744ba30643d47a70 (10.1.1.0/24)	VPC-B (ap-south-1a)	N/A	Disabled
ewb-igw-a	Internet Gateway	igw-085e7a...	Attached to VPC-A	N/A	N/A
Peering-A-to-B	VPC Peering	pcx-07068ebfe41b3d52d	Active (VPC-A ↔ VPC-B)	N/A	N/A
Public-RT-A	Route Table	rtb-038cf68d8165c9082	Public-Subnet-A	N/A	N/A
Private-RT-B	Route Table	rtb-01eb42cb035e40d07	Private-Subnet-B	N/A	N/A
EC2-Public-A (Bastion)	EC2 (t3.micro)	i-0a2b3c...	Public-Subnet-A	10.0.1.239	13.233.38.136
EC2-Private-B (App-B)	EC2 (t3.micro)	i-0d4e5f...	Private-Subnet-B	10.1.1.82	**None**



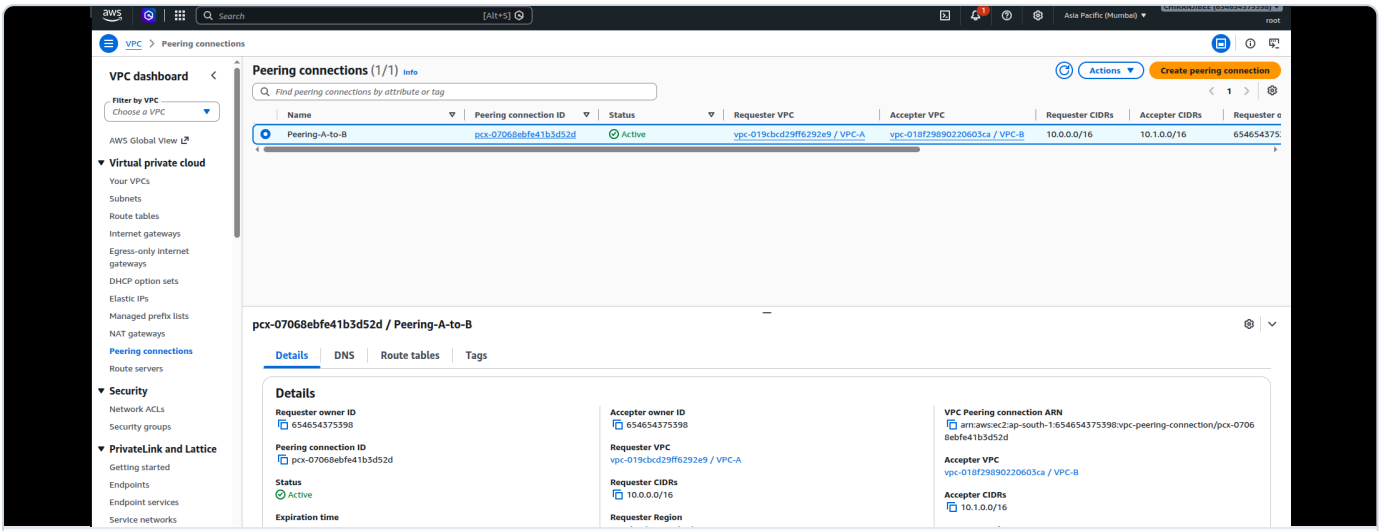
Evidence 2.1: VPC Console Overview: Host: AWS VPC Console | Test: VPC Availability | Result: Both `VPC-A` (`10.0.0.0/16`) and `VPC-B` (`10.1.0.0/16`) in Available state.



Evidence 2.2: Subnets Configuration: Host: AWS Subnet Console | Test: Explicit Subnet Associations | Result: `Public-Subnet-A`, `Private-Subnet-A`, and `Private-Subnet-B` verified.

03 / VPC PEERING & BIDIRECTIONAL ROUTING ANALYSIS

The Active VPC Peering Link

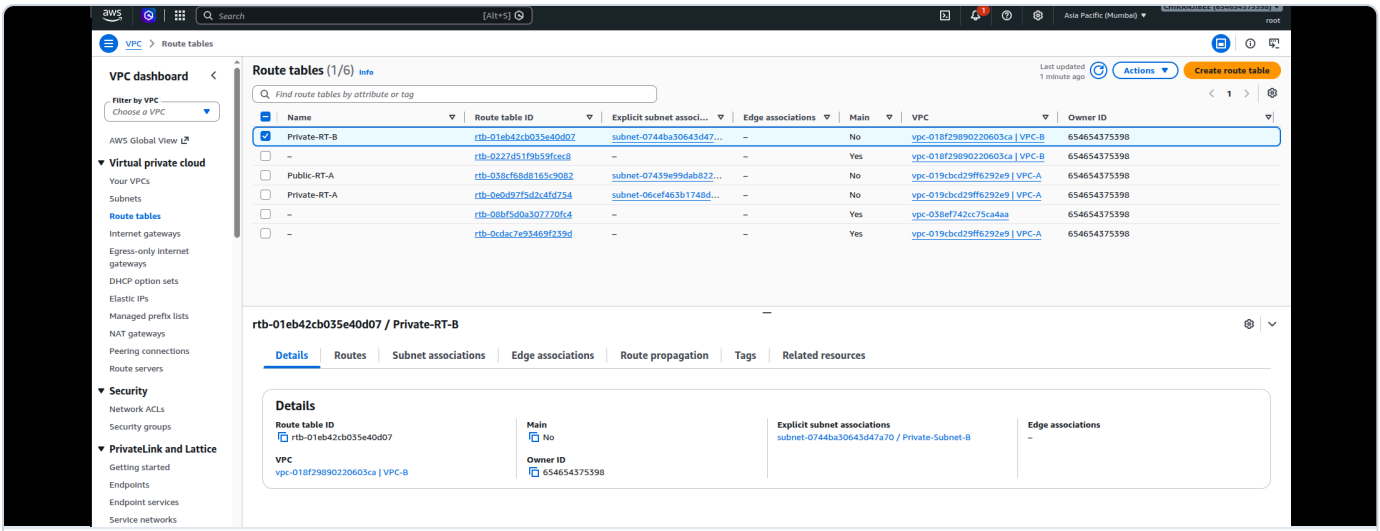


Evidence 3.1: Active VPC Peering: Host: VPC Peering Console | Test: Peering Status | Result: Connection `pcx-07068ebfe41b3d52d` Active between Requester `vpc-019cbcd29ff6292e9` and Accepter `vpc-018f29890220603ca`.

Route Tables: Forward Path vs. Return Path

An active peering connection alone does not route packets. Route tables must be updated symmetrically on both ends:

Route Table	Associated Subnet	Destination CIDR	Target	Purpose
`Public-RT-A` (VPC-A)	`Public-Subnet-A`	`10.0.0.0/16`	`local`	Intra-VPC routing within VPC-A
		`0.0.0.0/0`	`ewb-igw-a`	Internet ingress/egress for Bastion host
		`10.1.0.0/16`	`pcx-07068ebfe41b3d52d`	**Forward Path:** Bastion jumps traffic to VPC-B
`Private-RT-B` (VPC-B)	`Private-Subnet-B`	`10.1.0.0/16`	`local`	Intra-VPC routing within VPC-B
		`10.0.0.0/16`	`pcx-07068ebfe41b3d52d`	**Return Path:** Workload replies back to VPC-A



Evidence 3.2: Route Table Configuration: Host: Route Table Console | Test: Explicit Associations | Result: Route table `Private-RT-B` associated with `Private-Subnet-B`.

Key Rule: Symmetrical Routing — IP networking is always a two-way conversation. When App-A sends an ICMP Echo Request to App-B, the forward path is resolved via VPC-A's route table. If VPC-B does not possess a reverse route pointing to `pcx`, App-B's TCP/ICMP stack has no route to host, and the reply packet is dropped at the source kernel.

04 / SECURITY ARCHITECTURE & KEYLESS SSH PROXYJUMP

Inbound Security Group Rules

Security groups were configured using the principle of least privilege. Inbound access is scoped exclusively to individual lab hosts using `/32` CIDRs:

Security Group	Associated Instance	Protocol / Port	Allowed Source	Design Rationale
`SG-Public-Bastion`	`EC2-Public-A` (Bastion)	SSH / TCP 22	`Admin Public IP/32`	Restricts internet administration to student workstation only
`SG-Private-Test`	`EC2-Private-B` (App-B)	SSH / TCP 22	`10.0.1.239/32` (Bastion IP)	Allows administrative jump access exclusively from Bastion
`SG-Private-Test`	`EC2-Private-B` (App-B)	Custom ICMP (Type 8, Code 0)	`10.0.0.0/16` (VPC-A CIDR)	Permits ping connectivity testing across the peering connection

Keyless Remote Administration via OpenSSH ProxyJump

Security Anti-Pattern: Storing the private key (`.pem`) on a public bastion host creates an immediate lateral movement vulnerability if the bastion is compromised.
Cloud Best Practice: Keep the private key strictly on the student laptop, and use OpenSSH `ProxyJump`:

```
# LOCAL LAPTOP CONFIGURATION: ~/.ssh/config

Host ewb-bastion
    HostName 13.233.38.136
    User ubuntu
    IdentityFile ~/.ssh/ewb-lab.pem
    IdentitiesOnly yes

Host ewb-app-b
    HostName 10.1.1.82
    User ubuntu
    IdentityFile ~/.ssh/ewb-lab.pem
    IdentitiesOnly yes
    ProxyJump ewb-bastion
```

With this configuration, executing `ssh ewb-app-b` from the local machine transparently tunnels through the Bastion over private IP space. The Bastion acts solely as a TCP stream proxy (`-W`), and no credentials touch cloud disks.

05 / VALIDATION, TEST RESULTS & CONTROLLED FAILURE

Verification Test Summary

Test Index	Test Type	Host & Execution Command	Observed Result	Status
01 / Baseline	Pre-Peering Isolation	Inside `EC2-Public-A`: `ping -c 4 10.1.1.82`	100% packet loss (no route to host)	PASSED
02 / Forward Path	ICMP Reachability	Inside `EC2-Public-A`: `ping -c 4 10.1.1.82`	4/4 replies; 0% loss; RTT avg = 0.738ms	PASSED
03 / Reverse Path	Bidirectional Return	Inside `EC2-Private-B`: `ping -c 4 10.0.1.239`	4/4 replies; 0% loss; verified symmetrical routing	PASSED
04 / ProxyJump SSH	Private SSH Access	From Local Laptop: `ssh ewb-app-b`	Terminal shell on App-B achieved via Bastion	PASSED
05 / Workload Isolation	No Public Exposure	AWS EC2 Console instance details inspection	Public IPv4 field empty for `EC2-Private-B`	PASSED

Real Terminal Evidence: ICMP Ping Verification

```
ubuntu@ip-10-0-1-239:~$ ping -c 4 10.1.1.82
PING 10.1.1.82 (10.1.1.82) 56(84) bytes of data.
64 bytes from 10.1.1.82: icmp_seq=1 ttl=64 time=1.96 ms
64 bytes from 10.1.1.82: icmp_seq=2 ttl=64 time=0.296 ms
64 bytes from 10.1.1.82: icmp_seq=3 ttl=64 time=0.274 ms
64 bytes from 10.1.1.82: icmp_seq=4 ttl=64 time=0.418 ms

--- 10.1.1.82 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3072ms
rtt min/avg/max/mdev = 0.274/0.738/1.964/0.709 ms
ubuntu@ip-10-0-1-239:~$
```

Evidence 5.1: ICMP Ping Success: Host: `ubuntu@ip-10-0-1-239` | Test: `ping -c 4 10.1.1.82` | Result: 4 packets transmitted, 4 received, 0% packet loss, 0.738ms avg RTT.


```
ubuntu@ip-10-0-1-239:~$ ping -c 4 10.1.1.82
PING 10.1.1.82 (10.1.1.82) 56(84) bytes of data.
64 bytes from 10.1.1.82: icmp_seq=1 ttl=64 time=1.96 ms
64 bytes from 10.1.1.82: icmp_seq=2 ttl=64 time=0.296 ms
64 bytes from 10.1.1.82: icmp_seq=3 ttl=64 time=0.274 ms
64 bytes from 10.1.1.82: icmp_seq=4 ttl=64 time=0.418 ms

--- 10.1.1.82 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3072ms
rtt min/avg/max/mdev = 0.274/0.738/1.964/0.709 ms
```


Controlled Failure & Security Group Resilience Test

1. **Failure Injection:** Removed the inbound ICMP Echo request rule from `SG-Private-Test` for App-B.
2. **Observation:** Rerunning `ping -c 4 10.1.1.82` from App-A immediately produced 100% packet loss, proving that the active peering connection cannot bypass security groups.
3. **Recovery:** Reinstated the ICMP rule. Pings immediately resumed with 0% packet loss without requiring an instance restart or route reset, proving that Security Groups are **stateful** and evaluate rules dynamically.

06 / STUDENT VIVA VOCE & TECHNICAL DEFENSE

Below are the comprehensive explanations for the four core evaluation questions specified in Page 12 of the EWB Student Workbook:

1. Why is the return route needed?

> **Technical Answer:** IP routing is connection-oriented and stateless at the route table layer. Even though VPC-A has a route directing packets destined for `10.1.0.0/16` into `pcx-xxxx`, App-B must craft an IP reply packet with destination address `10.0.1.239`. When App-B's OS kernel hands the packet to the VPC virtual router, the router evaluates VPC-B's route table. If there is no route for `10.0.0.0/16` targeting `pcx-xxxx`, the router drops the packet immediately as unroutable.

2. Why is the App-B SSH source the bastion private IP?

> **Technical Answer:** When using SSH `ProxyJump`, your local laptop establishes an SSH transport connection to the Bastion's public IP. Once authenticated, OpenSSH instructs the Bastion to open a separate, direct TCP connection from itself to App-B's private IP (`10.1.1.82:22`). Because this second connection originates directly from the Bastion instance, the source IP observed in the IP header by App-B's network interface is the Bastion's primary private IP (`10.0.1.239`), not your laptop's public IP.

3. Why does Active peering not guarantee a successful ping?

> **Technical Answer:** The "Active" state merely indicates that AWS has provisioned the virtual peering link between the two VPCs' SDN infrastructures. A successful ping requires three distinct layers to be properly configured:
>
> 1. **Routing Layer:** Both VPC route tables must have explicit routes to the peer CIDR.
> 2. **Security Layer (Security Groups):** Destination instance must explicitly allow inbound ICMP Echo Request (Type 8, Code 0) from the source IP/CIDR.
> 3. **Subnet ACL Layer (NACLs):** Subnet Network ACLs must permit ephemeral and ICMP return traffic in both directions.

4. Can VPC B use VPC A's internet gateway over peering?

> **Technical Answer: No.** AWS VPC Peering is strictly **non-transitive**. AWS explicitly blocks edge-to-edge routing over peering connections. A VPC peering connection can only route traffic whose destination IP falls within the paired VPC's CIDR block. Traffic cannot transit through a peered VPC to reach the internet, a virtual private gateway, or another peering connection. (For transitive routing, AWS Transit Gateway must be used).

07 / RESPONSIBLE RESOURCE CLEANUP & KEY LEARNINGS

Resource Teardown Checklist (Executed)

To finish responsibly and eliminate cloud costs (Page 14 of Workbook):

1. **Compute:** Terminated instances `EC2-Public-A` and `EC2-Private-B`. Confirmed attached EBS root volumes were deleted on termination.
2. **Elastic IPs:** Released allocated public IPv4 addresses to stop idle IP charges.
3. **Peering & Routing:** Deleted VPC Peering connection `pcx-07068ebfe41b3d52d` (which automatically invalidated the peer routes in both route tables).
4. **Security Groups & Subnets:** Deleted custom security groups `SG-Public-Bastion` and `SG-Private-Test` after ENIs detached. Removed subnet associations and deleted custom subnets.
5. **Gateways & VPCs:** Detached and deleted Internet Gateway `ewb-igw-a`, and terminated `VPC-A` and `VPC-B`.
6. **Billing Verification:** Confirmed 0 active billable instances or NAT gateways remained in `ap-south-1`.

Student Reflection & Key Takeaways

- **Network Isolation is King:** Keeping application workloads in private subnets without public IPs eliminates 99% of external automated scan attacks.
- **Route Tables are Deterministic:** AWS always selects the most specific prefix (longest prefix match). An explicit `/16` route to a peering connection will always take priority over a `0.0.0.0/0` default route.
- **Security in Depth:** Layering stateful Security Groups on top of route tables and private addressing guarantees that even if a host is reachable, only permitted protocols can communicate.

LEARN. ELEVATE. EMPOWER.

AWS Solutions Architect Associate Portfolio Project · Subhransu Sekhar Lenka · Oct 2026